

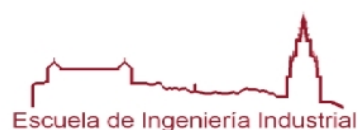
Práctica 3: Programación de Arduino.

Primeros ejemplos.

Escuela de Ingeniería Industrial de Toledo

Autor

Juan Moreno García



Índice general

3. Programación de Arduino	2
3.1. Objetivos.	2
3.2. Dos primeros ejemplos.	2
3.2.1. Primera modificación de <i>Blink</i>	2
3.2.2. Segunda modificación de <i>Blink</i> .	3
3.3. Uso de la función <code>analogWrite()</code> .	5
3.4. Uso de la funciones <code>digitalRead()</code> y <code>digitalWrite()</code> .	5
3.5. Uso de la funciones <code>analogRead()</code> y <code>analogWrite()</code> .	7
3.6. Uso de receptor infrarojo y mando infrarojo	10
3.6.1. Añadir carpeta <code>IRremote</code> en Arduino	10
3.6.2. Mando infrarrojo y Monitor Serial	11
3.6.3. Mando infrarrojo y Monitor Serial: Versión mejorada	13
3.7. Uso del <code>lcd</code>	14
3.7.1. Hola mundo / Mundo hola	15
3.7.2. Posicionando el cursor en el <code>lcd</code>	16
3.7.3. Utilizando el <code>lcd</code> y el mando IR conjuntamente	17

3

Programación de Arduino

3.1. Objetivos.

Esta práctica pretende acercar al alumno a la programación de Arduino, pero se utilizará un *nodeMCU* por su precio y sus posibilidades para la Wifi. Por ello, se desarrollarán todos los conceptos necesarios para la realización de los primeros programas que utilicen sensores y actuadores. Al final de la práctica el alumno debe ser capaz de:

1. Crear programas sencillos que utilicen sensores y actuadores elementales.
2. Moverse por la página web de Arduino para buscar información sobre los elementos necesarios.

3.2. Dos primeros ejemplos.

Para que el *nodeMCU* pueda ejecutar un sketch, que es como Arduino llama a sus programas, es necesario compilar, y posteriormente, cargar el código compilado. Por ello, primeramente se va a cargar un programa que utiliza el led integrado en el *nodeMCU*. El IDE de Arduino trae un Sketch de ejemplo de uso del led de *Arduino* llamado *Blink* y que será la base de los dos ejemplos de esta sección.

3.2.1. Primera modificación de *Blink*

La primera modificación de *Blink* permite encender y apagar el led integrado en el *nodeMCU* enviando información al Monitor Serie.

```
#define LED 2 // conectado al led integrado en nodeMCU

void setup() {
    pinMode(LED, OUTPUT); // LED como salida
    Serial.begin(115200);
}

void loop() {
    digitalWrite(LED, LOW);    // enciende el LED
    Serial.println("Encendido 1 segundo");
    delay(1000);               // espera 1 segundo
    digitalWrite(LED, HIGH);   // apaga el LED
    Serial.println("Apagado 3 segundos");
    delay(3000);               // espera 3 segundos
}
```

Para realizar la prueba realizar los siguientes pasos:

1. Abrir el IDE de Arduino con la configuración correcta para el nodeMCU (ya se ha explicado). Copiar el programa anterior en el IDE.
2. Pulsar el botón Verificar o el menú *Sketch>Verificar/Compilar* o *control r* (también hay un botón directo en el IDE). Las tres acciones hacen lo mismo, compilar el programa.
3. Si todo ha ido correctamente, es decir, una compilación sin errores, se debe cargar el programa en el *nodeMCU*. Para esto el *nodeMCU* debe estar conectado al PC, y seleccionados correctamente la placa y el puerto. Esto debe ocurrir sin ningún incidente.
4. Una vez realizado esto el *nodeMCU* ejecutará la aplicación cargada, con lo que el LED integrado parpadeará alternativamente.

El led integrado de la placa está conectado al pin 2. El programa utiliza la directiva *define* para declarar en número de pin asociado llamado *LED* que identificará al pin de salida en este programa. La función *setup()* utiliza la orden (*pinMode(LED, OUTPUT)*) para asignar el pin 2 (*LED*) como de salida (*OUTPUT*), y además inicializa el puerto a una velocidad de 115200 baudios. El *loop()* se ejecuta reiteradamente encendiendo el led durante 1 segundo, poniendo LOW en dicho pin, y enviando al Monitor Serie el texto “Encendido 1 segundo”, y finalmente, apaga el led durante 3 segundos, poniendo HIGH en el pin, y enviando el mensaje “Apagado 3 segundos” al Monitor Serie.

3.2.2. Segunda modificación de *Blink*.

La siguiente modificación consistirá en conectar a un led externo utilizando la patilla *D1 (GPIO5)*. El hardware requerido es el siguiente:

1. Placa *nodeMCU*.
2. Placa de pruebas (Breadboard).
3. Un led.

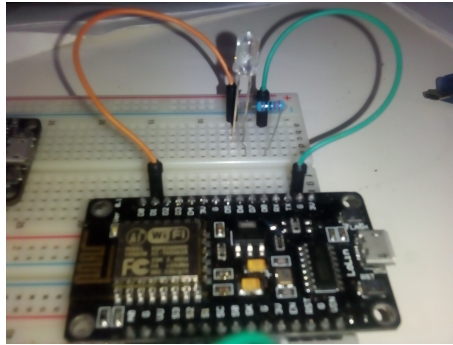


Figura 3.1: Circuito para la segunda modificación de *Blink*.

4. Una resistencia de 220 ohms.

La figura 3.1 muestra las conexiones a realizar. Se conecta el ánodo (positivo), el extremo más largo del led, al pin *D1* del *nodeMCU* a través de una resistencia de 220 ohms. Conectar el cátodo (negativo), la pata más corta del led, directamente a tierra (*Ground*).

El programa es el siguiente:

```
#define LED 5 // pin D1 del nodeMCU

void setup() {
    pinMode(LED, OUTPUT); // LED como salida
    Serial.begin(115200);
}

void loop() {
    digitalWrite(LED, HIGH); // enciende el LED
    Serial.println("Encendido 1 segundo");
    delay(1000); // espera 1 segundo
    digitalWrite(LED, LOW); // apaga el LED
    Serial.println("Apagado 3 segundos");
    delay(3000); // espera 3 segundos
}
```

El programa es muy similar al anterior, siendo las diferencias las siguientes:

- El pin utilizado es el *D1* (*GPIO5*), luego la directiva *define* cambia.
- Al ser el led externo se enciende poniéndolo a *HIGH* y se apaga a *LOW*.

La figura 3.1 muestra cómo realizar el montaje en una placa de pruebas (protoboard).

3.3. Uso de la función `analogWrite()`.

Para mostrar la utilización de la función `analogWrite()` se presenta un sketch que produce la atenuación de un led, que es una traducción y adaptación del sketch *Fade* de los ejemplos de Arduino. `analogWrite()` utiliza la modulación por ancho de pulso (PWM) para crear un efecto de atenuación, para ello, varía el valor de salida de un pin digital desde su valor más bajo (0) hasta su valor más alto (255) y luego a la inversa.

El código es el siguiente:

```
#define led 5
int brillo = 0; // cuanto brilla el led
int incremento = 5; // incremento/decremento en el brillo

void setup() {
    pinMode(led, OUTPUT);
}

void loop() {
    // Asigna el brillo al led
    analogWrite(led, brillo);
    // cambia la dirección de la modificación
    if (brillo == 0 || brillo == 255) {
        incremento = -incremento ;
        delay(250); // Retardo para ver que llega al mínimo y al máximo
    }
    // modifica el brillo para la siguiente iteración
    brillo += incremento;
    // espera para poder ver el decremento
    delay(30);
}
```

Primeramente se declara la directiva `define led` que manda la salida analógica por el pin 5, donde estará conectado el led. A continuación se declaran las variables enteras *brillo* y *incremento* que contienen el brillo del led y el incremento en el brillo en cada ejecución de la función `loop()`.

A continuación aparece la función `setup()` que declara el pin 5 de salida, y es por donde el programa enviará la salida analógica al led.

La función `loop()` primero envía el valor de brillo, inicialmente a 0. A continuación, si se ha llegado a uno de los valores límite (0 o 255) se invierte la dirección de la atenuación, hacia delante o hacia atrás (*incremento* positivo o negativo respectivamente). Después se produce el incremento de la variable *brillo* en la cantidad contenida en la variable *incremento* (5 en este ejemplo).

3.4. Uso de la funciones `digitalRead()` y `digitalWrite()`.

El hardware necesario para este ejemplo es el siguiente:

1. Placa Arduino Uno.
2. Un pulsador.
3. Una resistencia de 10 K ohms.
4. Una placa de pruebas.

La figura 3.2 muestra el montaje. **CAMBIAR IMAGEN CON nodeMCU.**

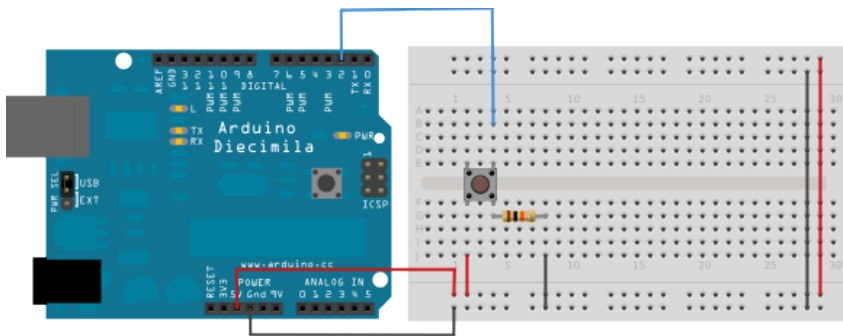


Figura 3.2: Uso de entrada y salida digital.

El código es el siguiente:

```
#define boton 4 // Pin del boton
#define led 5 // Pin del led

bool encendido; // true si led encendido

void setup() {
    pinMode(led, OUTPUT);
    digitalWrite(led, LOW); // Led apagado
    encendido = false;
    pinMode(boton, INPUT);
    Serial.begin(115200);
}

void loop() {
    int estado; // variable para leer el estado del boton
    estado = digitalRead(boton);
    // si boton pulsado es HIGH
    if (estado == HIGH) {
        digitalWrite(led, HIGH);
        if (!encendido) {
            Serial.println("Led encendido");
            encendido = true;
        }
    }
}
```

```

    } else {
        digitalWrite(led, LOW);
        if (encendido) {
            Serial.println("Led apagado");
            encendido = false;
        }
    }
}

```

Inicialmente el programa declara directivas *define* para identificar los pines digitales del pulsador y del led (pines 4 y 5 respectivamente).

A continuación, declara la variable *encendido* de tipo *bool* que estará a cierto si está encendido el led, o a falso en caso contrario. Se utiliza para también mostrar los mensajes de encendido o apagado una única vez.

Después, la función *setup()* inicializa el pin del led como de salida, y lo mantiene apagado colocando a *LOW* dicho pin. La variable *encendido* la inicializa a *false*, el pin del pulsador lo define como de entrada y fija la velocidad del puerto RS-232 a 115200 baudios.

La función *loop()* primero lee el valor de entrada del pulsador mediante la función *digitalRead()* y almacena el valor de lectura en *estado*. A continuación comprueba si el valor es *HIGH* o *LOW* invocando a la función *digitalWrite()* para encender (*digitalWrite(ledPin, HIGH)*) o apagar el led (*digitalWrite(ledPin, LOW)*) respectivamente. En ambos casos comprueba la variable *encendido* para mostrar el mensaje e invertir su valor en caso de ser la primera vez.

3.5. Uso de la funciones `analogRead()` y `analogWrite()`.

Este ejemplo muestra el uso de la funciones *analogRead()* y *analogWrite()*. Más concretamente, explica cómo leer un pin de entrada analógico, mapear el resultado en un rango desde 0 a 255, y entonces utilizar el resultado para asignar el “pulso de modulación de salida” de un pin de salida para oscurecer o iluminar un LED.

El hardware requerido es el siguiente:

- Placa nodeMCU.
- Un potenciómetro.
- Un led.
- Una resistencia de 220 ohms.

El montaje del circuito se muestra en la Figura 3.3.

Código

```

#define analogInPin A0 // pin para el potenciómetro
#define analogOutPin 5 // pin para el LED

int ultimoMapeo = 0; // contiene el valor anterior de mapeado

```

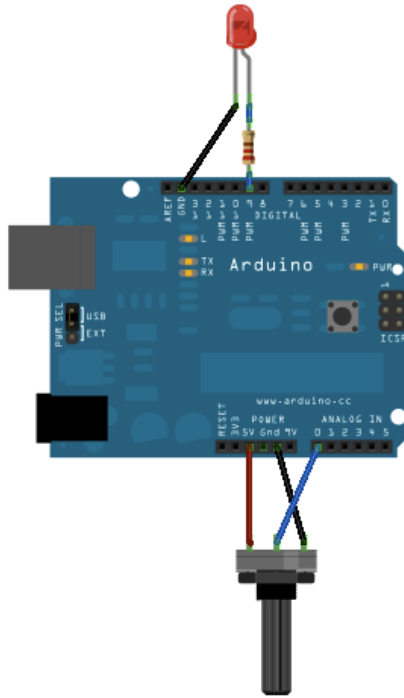


Figura 3.3: Uso de entrada y salida analógica **CAMBIAR A NODEMCU.**

```
void setup() {
    Serial.begin(115200);
    // se inicializa ultimoMapeo
    ultimoMapeo = map( analogRead(analogInPin) , 0, 1023, 0, 255);
}

void loop() {
    int sensorV = 0; // valor leído de potenciómetro
    int mapeado = 0; // valor de salida al pin (analógica)
    // lectura del pin analógico
    sensorV = analogRead(analogInPin);
    mapeado = map(sensorV, 0, 1023, 0, 255);
    if (mapeado!=ultimoMapeo) {
        // escribe el valor mapeado ya que la salida toma valores
        // entre 0-255
        analogWrite(analogOutPin, mapeado);
        // envia el resultado al Monitor Serie
        Serial.print("sensor = ");
        Serial.print(sensorV);
        Serial.print("\t ultimoMapeo = ");
        Serial.print(ultimoMapeo);
    }
}
```

```

        Serial.print("\t output = ");
        Serial.println(mapeado);
        ultimoMapeo = mapeado;
    }
    // espera 2 milisegundos antes del siguiente ciclo para que el
    // convertidor analógico a digital se estabilice
    delay(2);
}

```

Inicialmente el programa utiliza la directiva *define* para identificar los pines analógicos de salida del potenciómetro y del led (pines A0 y 5 respectivamente).

Después, la función *setup()* inicializa las comunicaciones serie a 115200 baudios y la variable *ultimoMapeo* al valor actual *mapeado*.

La función *loop()*, mediante la sentencia *sensorV = analogRead(analogInPin)*, lee la entrada analógica, a continuación mapea la salida al rango de salida utilizando para ello la función *map()* y asigna el valor a la variable *mapeado*. Esto es necesario para realizar un cambio de escala entre 0-1023 a 0-355, valor válido para el led. Al final de este texto se explica cómo funciona esta función.

A continuación, cambia el valor de salida al valor calculado en la variable *mapeado* si ha habido cambio en la lectura, y muestra el resultado en el *Monitor Serie* del IDE de Arduino.

Finalmente, espera dos milisegundos antes de la ejecución del siguiente bucle para repetir el proceso. Esto da tiempo a la realización de la conversión analógico-digital.

Función `map(value, fromLow, fromHigh, toLow, toHigh)`

Descripción:

Realiza un cambio (mapeo) de rango entre dos intervalos, de un intervalo a otro. Ésto significa que, un valor con respecto al rango $[fromLow - fromHigh]$ será mapeado al rango $[toLow - toHigh]$.

La función maneja correctamente también los números negativos, por ejemplo, el código `y = map(x, 1, 50, 50, -100)` funciona correctamente.

La función *map()* usa matemática de enteros por lo que no generará números decimales, siendo éstos truncados (no son redondeados).

Parámetros:

- *value*: Número a cambiar de rango (mapear).
- *fromLow*: Límite inferior del rango actual.
- *fromHigh*: Límite superior del rango actual.
- *toLow*: Límite inferior del rango deseado.
- *toHigh*: Límite superior del rango deseado.

3.6. Uso de receptor infrarojo y mando infrarojo

En esta sección se mostrará cómo conectar un mando infrarrojo a un *nodeMCU* y se verán algunos ejemplos.

3.6.1. Añadir carpeta IRremote en Arduino

Para que funcione el receptor infrarrojo es necesario instalar la librería *IRremoteESP8266* necesaria para trabajar con el *nodeMCU*. Un proceso parecido se tiene que hacer para que funcione en *Arduino*. Se debe hacer lo siguiente:

Bajar de la dirección <https://github.com/markszabo/IRremoteESP8266> el archivo zip que contiene la librería y copiar la carpeta que se obtiene al descomprimir con el nombre “IRremoteESP8266” en la subcarpeta *libraries* de la carpeta de instalación del IDE de Arduino.

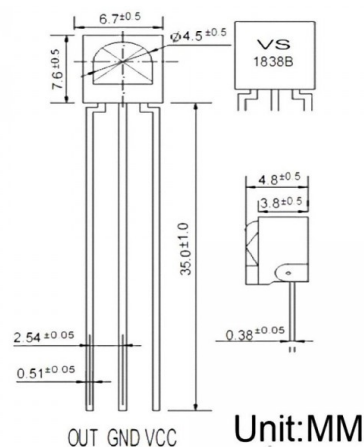


Figura 3.4: Conexiones del VS1838B.

Para probar el correcto funcionamiento es necesario un receptor infrarrojo, por ejemplo, el VS1838B. Se debe conectar la primera conexión al pin del *nodeMCU* (en el ejemplo se usa el pin 4), la segunda a tierra (GND) y la última a 3 voltios (VCC) como muestra la Figura 3.4. Para probar su correcto funcionamiento se debe cargar el siguiente programa en el IDE de Arduino:

```
#include <IRremoteESP8266.h>
#include <IRrecv.h>

int RECV_PIN = 4; // Pin digital al que se ha conectado el receptor IR

IRrecv irrecv(RECV_PIN); // Crea el objeto para manejar el infrarojo

decode_results results; // struct donde se recibe el resultado recibido
```

```

void setup() {
    Serial.begin(115200); // Inicia RS232 para el Monitor Serial
    irrecv.enableIRIn(); // Inicia el receptor
}
void loop() {
    if (irrecv.decode(&results)) {
        // Valor leído al Monitor Serial
        Serial.println((int)results.value, HEX); // (int) obligatorio
        irrecv.resume(); // Recibe el siguiente valor
    }
}

```

Este programa devuelve el código pulsado del mando infrarrojo por el monitor serie. Además, se puede utilizar para obtener los códigos que devuelven cada uno de los botones de un mando a distancia, siendo útil para la realización de una aplicación que trabaje con el mando del que se desconoce la codificación de sus botones.

3.6.2. Mando infrarrojo y Monitor Serial

A continuación se muestra un programa que ante una pulsación de un botón del mando devuelve una cadena de texto que identifica dicho botón y la muestra por el monitor serie. El programa es muy similar al anterior excepto por el hecho de que se le ha añadido la función *translateIR()* que devuelve el *String* que identifica al botón pulsado cuando se produce una.

```

#include <IRremoteESP8266.h>
#include <IRrecv.h>

int RECV_PIN = 4; // Pin digital al que se ha conectado el receptor IR

IRrecv irrecv(RECV_PIN); // Crea objeto de 'irrecv'
decode_results results; // Crea objeto de 'decode_results'

void setup() {
    Serial.begin(115200);
    irrecv.enableIRIn(); // Inicializa el receptor
}

void loop() {
    String s;
    if (irrecv.decode(&results)) { // Si se ha recibido alguna señal
        s = translateIR(); // Muestra informacion en LCD
        if (s!="") {
            Serial.println( "[" + s + "]" );
        }
        irrecv.resume(); // Recibir el siguiente valor
    }
}

```

```
String translateIR() {
    String s;
    unsigned long r = results.value;

    if (r==0xFFA25D) {
        s = "ON/OFF";
    } else if (r==0xFF629D) {
        s = "MODE";
    } else if (r==0xFFE21D) {
        s = "Vo1OFF";
    } else if (r==0xFF22DD) {
        s = "PLAY/PAUSE";
    } else if (r==0xFF02FD) {
        s = "PREV";
    } else if (r==0xFFC23D) {
        s = "NEXT";
    } else if (r==0xFFE01F) {
        s = "EQ";
    } else if (r==0xFFA857) {
        s = "-";
    } else if (r==0xFF906F) {
        s = "+";
    } else if (r==0xFF6897) {
        s = "0";
    } else if (r==0xFF9867) {
        s = "LOOP";
    } else if (r==0xFFB04F) {
        s = "U/SD";
    } else if (r==0xFF30CF) {
        s = "1";
    } else if (r==0xFF18E7) {
        s = "2";
    } else if (r==0xFF7A85) {
        s = "3";
    } else if (r==0xFF10EF) {
        s = "4";
    } else if (r==0xFF38C7) {
        s = "5";
    } else if (r==0xFF5AA5) {
        s = "6";
    } else if (r==0xFF42BD) {
        s = "7";
    } else if (r==0xFF4AB5) {
        s = "8";
    } else if (r==0xFF52AD) {
        s = "9";
    } else {
        s = ""; // Rebote, se devuelve ""
    }
    return String(s);
}
```

De este programa es importante destacar cómo se ha utilizado el *String* en la función *translateIR()*. La función devuelve un objeto *String*, primero calcula el *String* a devolver y luego invoca al constructor del objeto mediante la orden *String(s)* que crea al mismo. El concepto utilizado aquí es similar al uso del *malloc()*, para devolver un *String* se debe crear y posteriormente devolver con la invocación a un constructor mediante la orden *return*, lo que hace que el programa que lo invocó obtenga la referencia.

Este programa tiene un problema: se utilizan 50 líneas para seleccionar el *String* resultado. El código se puede mejorar para ser más corto y efectivo.

3.6.3. Mando infrarrojo y Monitor Serial: Versión mejorada

Este programa realiza la misma función que el anterior, pero el código fuente de la función *translateIR()* ha sido mejorado, presentando una función que lo acorta, y que ayuda a una posible modificación de los valores a trabajar. El programa hace uso de dos vectores que contienen la palabra clave (el código del botón) y la cadena a tratar denominados *claves* y *valor* respectivamente. El programa es el siguiente:

```
#include <IRremoteESP8266.h>
#include <IRrecv.h>

unsigned long claves[21] = {0xFFA25D, 0xFF629D, 0xFFE21D, 0xFF22DD, 0xFF02FD,
    0xFFC23D, 0xFFE01F, 0xFFA857, 0xFF906F, 0xFF6897, 0xFF9867, 0xFFB04F,
    0xFF30CF, 0xFF18E7, 0xFF7A85, 0xFF10EF, 0xFF38C7, 0xFF5AA5, 0xFF42BD,
    0xFF4AB5, 0xFF52AD};
String valor[21] = {"ON/OFF", "MODE", "VoLOFF", "PLAY/PAUSE", "PREV", "NEXT",
    "EQ", "-", "+", "0", "LOOP", "U/SD", "1", "2", "3", "4", "5", "6",
    "7", "8", "9"};

int RECV_PIN = 4; // pin 1 del receptor IR al pin 2 de Arduino

IRrecv irrecv(RECV_PIN); // Crea objeto de 'irrecv'
decode_results results; // Crea objeto de 'decode_results'

void setup() {
    Serial.begin(115200);
    irrecv.enableIRIn(); // Inicializa el receptor
}

void loop() {
    String s;
    if (irrecv.decode(&results)) { // Si se ha recibido alguna señal
        s = getValor(results.value); // Muestra informacion en LCD
        if(s!="") {
            Serial.println( "[" + s + "]" );
        }
        irrecv.resume(); // Recibir el siguiente valor
    }
}
```



```
String getValor(unsigned long c) {  
    byte i;  
    // Búsqueda del índice de acceso a la clave  
    i=0;  
    while( (i<21) && (claves[i]!=c) ) {  
        i++;  
    }  
    // Selección del valor de retorno  
    if (i==21) return String("");  
    else return String(valor[i]);  
}
```

La función *translateIR()* ha sido sustituida por una función llamada *getValor()* que devuelve el *String* que identifica el botón pulsado. El parámetro de la misma es el código del botón pulsado, que es buscado en el vector *claves* (bucle *while*), y cuando se encuentra se utiliza su índice (variable *i*) para seleccionar el *String* correcto de *valor* (vector de *String*). Si no se ha encontrado (*i*==21) se devuelve un *String* vacío.

3.7. Uso del lcd

En esta sección se mostrará cómo conectar un lcd a un *nodeMCU* mediante el protocolo *I2C*, para ello se utilizará un interfaz *I2C*. Se muestran tres ejemplos de su uso.



Figura 3.5: Interfaz I2C y lcd en la placa de pruebas.

3.7.1. Hola mundo / Mundo hola

El siguiente ejemplo muestra los mensajes “Practicas II(1)” y “Practicas II(0)” en las líneas 1 y 0 del lcd respectivamente. Para ello, es necesario un lcd de 16x2 (vale cualquier otro), un interfaz I2C para conectar el lcd al *nodeMCU* utilizando solamente dos pines. El lcd y el interfaz I2C deben estar enfrentados, como muestra la Figura 3.5, de tal forma que cada columna de la placa de pruebas conecta lo pines del lcd y del interfaz. Las salidas del interfaz I2C (GND, VCC, SDA y SCL) serán conectadas al *nodeMCU* de la siguiente forma:

- GND a cualquier G del *nodeMCU*.
- VCC a VU del *nodeMCU*. Esta salida del *nodeMCU* da 5 voltios de salida, necesarios para que el lcd se vea correctamente. Si se conecta a 3V se verá, pero levemente. La forma en que se muestra se puede regular con el potenciómetro integrado en el interfaz I2C.
- SDA se conectará al pin seleccionado del lcd, en nuestro caso el pin 5 (D1).
- SCL se conectará al pin seleccionado del lcd, en nuestro caso el pin 4 (D2).

Como puede verse, sólo se utilizan dos pines del *nodeMCU*.

Las librerías *LiquidCrystal_I2C.h* y *Wire.h* son necesarias para manejar el lcd y el interfaz I2C respectivamente. Es necesario añadir la librería *LiquidCrystal_I2C.h* al IDE de Arduino. Los pasos necesarios para añadir la librería son los siguientes, y son muy similares a los seguidos en la Sección 3.6.1 para añadir la librería *IRremote*:

Bajar de la dirección https://github.com/marcoschwartz/LiquidCrystal_I2C el archivo zip que contiene la librería y copiar la carpeta que se obtiene al descomprimir con el nombre “*LiquidCrystal_I2C.h*” en la subcarpeta *libraries* de la carpeta de instalación del IDE de Arduino.

El siguiente programa muestra el código a cargar en el *nodeMCU*.

```
#include <LiquidCrystal_I2C.h> // Necesaria para el lcd
#include <Wire.h> // Permite conectar los pines del interfaz I2C

LiquidCrystal_I2C lcd(0x27, 16, 2); // Crea objeto para lcd

byte turno; // Variable que selecciona fila a escribir

void setup() {
    turno = 1;
    Wire.begin(5, 4); // conecta el I2C a pines 5 y 4
    lcd.setBacklight(HIGH); // Activa luz del fondo del lcd
    lcd.begin(16,2); // Inicia lcd indicando n° de columnas y filas
    lcd.home(); // Coloca cursor en esquina superior izquierda del lcd
    lcd.setCursor(0,1); // Posiciona el cursor en lcd (col 1 y fila 0)
    lcd.print("Mundo hola (1)"); // Muestra texto en posicion cursor
}
```

```

void loop() {
    lcd.clear(); // limpia el lcd
    if (turno==1) {
        lcd.setCursor(0,1); // Posiciona el cursor en fila 1
        lcd.print("Mundo hola (1)"); // Muestra texto
        turno = 0; // Actualizar variable de cambio de fila
    } else {
        lcd.setCursor(0,0); // Posiciona el cursor en fila 0
        lcd.print("Hola mundo (0)"); // Muestra texto
        turno = 1; // Actualizar variable de cambio de fila
    }
    delay(2000);
}

```

El programa incluye las librerías *LiquidCrystal_I2C.h* y *Wire.h* necesarias para manejar el lcd y el interfaz I2C respectivamente.

A continuación se crea una variable del tipo *lcd* mediante la orden “*lcd(0x27, 16, 2)*”, donde el primer argumento depende del interfaz I2C utilizado (ver fabricante) y los dos siguientes parámetros indican el número de columnas y filas respectivamente del lcd.

La variable *turno* se utiliza para seleccionar la fila en la que se mostrará el mensaje. El *setup()* realiza las siguientes inicializaciones:

- La variable *turno* a 1 (primero se escribe en fila 1).
- Conecta las salidas SDA y SCL del interfaz a los pines 5 y 4 respectivamente mediante la orden *Wire.begin(5, 4)*.
- Activa la luz de fondo del lcd (*lcd.setBacklight(HIGH)*), inicia el lcd indicando que tiene 16 columnas y 2 filas (*lcd.begin(16,2)*), coloca cursor en esquina superior izquierda del lcd mediante la orden *lcd.home()*, posiciona el cursor en la columna 1 y la fila 0 (*lcd.setCursor(0,1)*), y muestra el mensaje “Mundo hola (1)” en dicha posición.

El bucle *loop()* muestra los mensajes “Mundo hola (1)” y “Hola mundo (0)” en la fila 1 y 0 respectivamente, mostrándolo durante 2 segundos. Utiliza la función *setCursor(columna, fila)* para colocar el cursor en el lcd y va asignando a *turno* el valor de la siguiente fila en la que toca mostrar.

3.7.2. Posicionando el cursor en el lcd

El siguiente programa desplaza un 0 en la fila 0 y un 1 en la fila 1 a lo largo de la misma alternativamente. Es bastante similar al anterior. Probarlo para ver su efecto y hacer modificaciones sobre él.

```

#include <LiquidCrystal_I2C.h>
#include <Wire.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

```

```

int fila = 0; // Indica fila a usar
int pos = 0; // Posicion dentro de la fila

void setup() {
    Wire.begin(5, 4); // Pines del nodeMCU a usar
    lcd.setBacklight(HIGH);
    lcd.begin(16,2);
    lcd.home();
    lcd.setCursor(pos, fila); // Posicion inicial del cursor
    lcd.print(""); // Mensaje inicial
}

void loop() {
    lcd.clear(); // Limpia LCD
    if (fila==0) { // Si fila 0
        lcd.setCursor(pos,0); // posiciona cursor
        lcd.print("0"); // muestra un 0
        fila = 1; // Cambio a fila 1
    } else {
        lcd.setCursor(pos,1); // posiciona cursor
        lcd.print("1"); // muestra un 1
        fila = 0; // Cambio a fila 1
        pos = (pos+1) % 16; // Incrementa y si es 16 pasa a 0
    }
    delay(1000); // Retardo para visualizar
}

```

3.7.3. Utilizando el lcd y el mando IR conjuntamente

El último programa muestra el uso conjunto del mando y receptor IR junto con el lcd en el nodeMCU. El montaje del lcd es el mismo que en el caso anterior (Sección 3.7.1), además el montaje del receptor y mando IR es igual que en la Sección 3.6, utilizando el pin 0 del *nodeMCU* en este caso. El siguiente código muestra el programa desarrollado:

```

#include <IRremoteESP8266.h>
#include <IRrecv.h>

#include <LiquidCrystal_I2C.h>
#include <Wire.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);

unsigned long claves[21] = {0xFFA25D, 0xFF629D, 0xFFE21D, 0xFF22DD, 0xFF02FD,
    0xFFC23D, 0xFFE01F, 0xFFA857, 0xFF906F, 0xFF6897, 0xFF9867, 0xFFB04F, 0xFF30CF,
    0xFF18E7, 0xFF7A85, 0xFF10EF, 0xFF38C7, 0xFF5AA5, 0xFF42BD, 0xFF4AB5,
    0xFF52AD};
String valor[21] = {"ON/OFF", "MODE", "Vo1OFF", "PLAY/PAUSE", "PREV", "NEXT",
    "EQ", "-", "+", "0", "LOOP", "U/SD", "1", "2", "3", "4", "5", "6", "7", "8",
    "9"};

```

```

int RECV_PIN = 0; // pin del receptor IR al pin 0 del nodeMCU

IRrecv irrecv(RECV_PIN); // Crea objeto de 'irrecv'
decode_results results; // Crea objeto de 'decode_results'

void setup() {
    Serial.begin(115200);
    irrecv.enableIRIn(); // Inicializa el receptor
    Wire.begin(5, 4);
    lcd.setBacklight(HIGH); //Use predefined PINS consts
    lcd.begin(16,2);
    lcd.home();
    lcd.setCursor(0,1); // Situa el cursor en 2ª Fila
    lcd.print("Practicas II"); // Muestra mensaje inicial
}

void loop() {
    String s;
    if (irrecv.decode(&results)) { // Si se ha recibido alguna señal
        s = getValor(results.value); // Recoge cadena desde código
        if(s!="") {
            Serial.println( "[" + s + "]" );
        }
        escribeLCD(s); // Escribo en LCD
        irrecv.resume(); // Recibir el siguiente valor
    }
}

void escribeLCD(String s) {
    lcd.clear(); // Limpia el LCD
    if(s!="") {
        lcd.setCursor(0,0); // Coloca el cursor en la fila 0
        lcd.print(s);
    } else {
        lcd.setCursor(0,1); // Coloca cursor en fila 1
        lcd.print( "--- Rebote ---"); // Muestra mensaje rebote
    }
    delay(200); // Retardo para evitar rebote
}

String getValor(unsigned long c) {
    byte i;
    i=0;
    while( (i<21) && (claves[i]!=c) ) {
        i++;
    }
    if (i==21) return String("");
    else return String(valor[i]);
}

```

Como puede verse, es una mezcla de los programas de la sección 3.6.3 y los desarrollados en esta sección. Además, se ha incorporado la salida por el monitor serie de la pulsación del botón del mando. Se ha reutilizado la función *getValor()* y se ha creado una nueva llamada *escribeLCD()*, lo que hace que la función principal *loop()* sea corta y más comprensible.